Chapter 11
Continuous Time Simulation

Andy Register

11.1 Introduction

Wind rustling the leaves, waves pounding the beach, the moon orbiting the earth, and a violin bow scraping across strings are a few examples of everyday events that carry on in a smooth, continuous way. Continuous modeling and simulation (M&S) attempts to reconstruct this familiar real-world flow inside an environment that allows us change conditions, make measurements, and otherwise study various aspects of the system. The basic reasons for simulating these continuous real-world events are similar to the reasons for M&S in general.

- The range of input conditions might be too dangerous (or too expensive) to apply to the real system.
- The real system might be slow to evolve making it impossible to test with a variety of conditions in a reasonable amount of time.
- The real system might be important to our well-being, like the atmosphere or the ocean, making it unwise to add a stimulus to the real system just to see what will happen.

Both continuous and event-driven M&S can be used to address these objectives; however, continuous M&S both develops and simulates those models in different and unique ways.

A. Register (✉)
Georgia Tech Research Institute, Atlanta, GA, USA
e-mail: andy.register@gtri.gatech.edu

M. L. Loper (ed.), *Modeling and Simulation in the Systems Engineering Life Cycle,*
Simulation Foundations, Methods and Applications,
DOI 10.1007/978-1-4471-5634-5_11

11.2 The Art and Science of Continuous Models

Today, with computers so pervasive in our lives, we naturally think about M&S in terms of computer models and computer programs. This was not always the case. Rapid advances in general-purpose computing between 1945 and 1955 ushered in a computer age where many analog techniques (Lundberg 2005) were replaced by computer-based, digital techniques. Computers proved to be very versatile in solving a wide range of problems; but not all. In some applications, analog techniques were already firmly established and did not quickly or easily give way to digital techniques. There are several examples where the accuracy or speed of computer-based models still lags behind the analog models. Wind tunnel testing is one good example. Even though tremendous advances in computer modeling have occurred, there is no substitute for putting a scale model inside a wind tunnel. It is also interesting to note that we call this activity testing rather than simulation.

Engineers use or have used a wide array of models and techniques to predict the result of applying various inputs to a system. In general, there are three ways to build a continuous model and perform continuous simulations:

- Scale models,
- Equivalent component continuous models, and
- Computer-based continuous models.

Scale models and equivalent component models are analog techniques while computer-based models belong in the class of digital techniques. The use of different techniques is important because in complicated applications, analog models sometimes support the development of computer models and vice versa. Using different modeling techniques gives us a way to cross-check answers and improves our ability to predict results. Successfully applying the right techniques to each new problem is part of the art and tremendous satisfaction that comes from working in the M&S field. Before jumping into the mathematics behind computer-based continuous models, a short historical perspective and some examples of the analog techniques are provided below.

11.2.1 Scale Models

Wind tunnel testing, automobile crash testing, tank testing, and so on substitute a physical scale model for the system under test and create carefully controlled test conditions. Sometimes the model is 1 to 1 scale even to the point of being a real, physical example of the system (e.g., automobile crash testing). The world's largest wind tunnel at the Ames Research Center has an $80' \times 120'$ cross section that can hold full-scale aircraft and helicopters for low and medium wind speed testing (Wikipedia Ames 2013). High-velocity wind tunnels and smaller, more accessible wind tunnels typically test with a physical model at a scale smaller than 1 to 1. In these tests, it is certainly valid to ask the question, "if you are using something other

than the actual aircraft surrounded by something other than its normal operational environment, is this a test or a simulation?" Before you try to answer that question, let us look at some other examples.

Similar to wind tunnel testing, scale models of a new ship design often undergo tank testing where hull models are subjected to various conditions (International Towing Tank Conference 2012). Long narrow tanks are used to measure drag and help determine the required engine size. Wider tanks are used to examine a ship's behavior in wind and waves coming from various directions at irregular intervals. Ice tanks are used to examine the behavior of ice-breaking hulls. For a variety of reasons, scale models are used for these investigations. Full-scale shipbuilding is expensive, so we often resort to scaled-down versions to examine the behavior of a new design. We also do not purposefully steer ships into category 5 hurricanes just to find out if they can survive the wind and waves. Instead, we build scale models and test them in relatively small tanks using simulated wind, waves, and ice: test or simulation?

Perhaps the largest and most intricate scale model ever constructed is the Mississippi Basin Model (Robinson 1992). In response to massive flooding in 1927, the Army Corps of Engineers promoted the idea of building a Mississippi River model and finally won approval to begin construction in 1943. Of course, computers were not yet available so a physical scale model was the only option. The model included most of the Mississippi River and its major tributaries; 1.25 million square miles of watershed, 41% of the continental USA, packed onto 200 acres near Clinton, Mississippi. An Army Corps of Engineers photo of a small section of the model is shown in (Fig 11.1). The model reproduced the drainage basin of the Mississippi River at a horizontal scale of 1:2000, a vertical scale of 1:100 and compressed a day of actual river flow into a little more than 7 min. The compressed time scale of the model allowed engineers to simulate the system more quickly than it naturally evolves. This allowed them to perform what-if analyses on various control options and pick the one with the best outcome. Eventually, computer modeling made the physical model obsolete and the Mississippi Basin Model was used for the last time during a major flood in 1973. The model is important for historical reasons along with the fact that during its operation, it successfully reduced property loss and saved countless lives.

The use of scale models is not limited to air or water flow. One of the most important parts of an automobile crash test is the crash-test dummy. We do not use real people in a crash test because it is too dangerous. Instead, we use a physical model of a person that attempts to simulate the body's weight, position, and dynamic behavior. Indeed, crash-test dummies have evolved significantly over time. We now have dummies of all sizes, even babies, with body frame sizes and weight distributions that model both males and females. Different dummies are required for low-speed versus high-speed crash tests, automobile versus aircraft tests, and for specialized crash tests for soldiers wearing body armor. While it is true that a crash test is indeed a test with respect to the automobile, the most meaningful safety data is collected using a simulated model of the automobile's human occupants.

Mississippi Basin Model
Vertical scale - 1:100; horizontal scale - 1:2000. Looking upstream on the Ohio River from Evansville. Indiana, Tennessee, and Cumberland Rivers are in the foreground showing the site of the Kentucky and Barkley Dams. Tradewater and Green Rivers are shown center. File No. 1270-4

Fig. 11.1 Mississippi basin model (U.S. Army 2013)

There are many other examples of scale models. Architects use them to gain perspective, mechanical engineers use them to find gears and screws that might interfere during assembly, civil engineers use them to make buildings safer during earthquakes, and aerospace engineers use them to develop rocket motors. Scale models are used to find the breaking point of nuclear reactor containment vessels and the stress points in structures. A scale model is a good alternative for designs that involve complicated processes: air flow, fluid flow, heat flow, wave action, fuel combustion, fires, explosions, vibrations, and so on. Most of these processes are very difficult to model mathematically. Equations can capture the gross behavior, but equations that correctly describe fine detail are difficult to determine and difficult to simulate. Scale models have their own shortcomings but mathematical difficulty is not among them. As computers and computer models have improved, many scale models have been taken over by computer-based M&S. We certainly expect this trend to continue; however, even if the use of scale models diminishes, it will not disappear completely.

Many engineers do not put scale models in the same class as other M&S techniques. This is somewhat unfortunate because the legacy of computer-based M&S lies in a rich history of scale model development. Many mathematical models were

derived and verified based on experience working with scale models. Scale models continue to improve our understanding of complicated processes. Computer model results are often anchored to data provided by sensors embedded in the scale model and the model's environment. Scale models form a bridge between M&S and T&E (test and evaluation), making scale models an essential part of both.

11.2.2 Equivalent-Component Continuous Models

Equivalent component models try to replace elements of the real-world system with components that produce an equivalent system that is easier to stimulate, monitor, change, and analyze. There are two subclasses of equivalent component models. The first subclass transforms the original problem into an equivalent mechanical system or electronic circuit. The transformation uses mechanical components like motors, masses, gears, springs, and dampers or electronic components like amplifiers, resistors, capacitors, and inductors to emulate mathematical relationships like addition, multiplication, and integration. The transformation's goal is to construct a mechanical or electronic model with the same mathematical equations as the original system or process. Note that unlike scale models, equivalent-component models begin with a mathematical description of the system.

The other subclass of equivalent-component modeling uses an appropriate substitute for system components that might be too expensive, dangerous, or politically incorrect. Examples of substitutes include a cheaper alternative for blood with the same viscosity, a safer alternative to sarin nerve gas with the same vapor pressure, and ballistics jelly with the same density and penetration behavior as a human or animal body. A crash-test dummy might also fall into this subclass. Indeed, substitutes and scale models are closely related because substitute materials are often used in conjunction with scale models. With substitutes, you usually have to make compromises in one or more properties to get a good match for the properties of interest. Substitutes are certainly important in niche applications; however, we will not discuss them further.

There are many ways to describe a continuous system: mathematical equations, state-space descriptions, transfer functions, bond graphs, and block diagrams, among others. For linear, stationary models, modern system theory provides a solid, mathematical foundation for transforming from one representation to another (Kailath 1980). For equivalent-component models, a block diagram representation is the most convenient because there are direct mechanical or electrical equivalents for gains, losses, summation, and integration. A quick example of simple linear motion will be used to illustrate the equivalent-modeling approach.

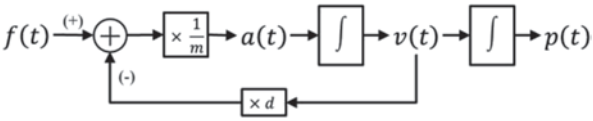
For simple motion, the fundamental equation is Newton's second law,

$$force = mass \times acceleration$$

Using Newton's equation and applying basic physics, we can draw the block diagram of the relationship between force and position shown in Fig. 11.2.

Fig. 11.2 Block diagram of linear motion with input force and output position

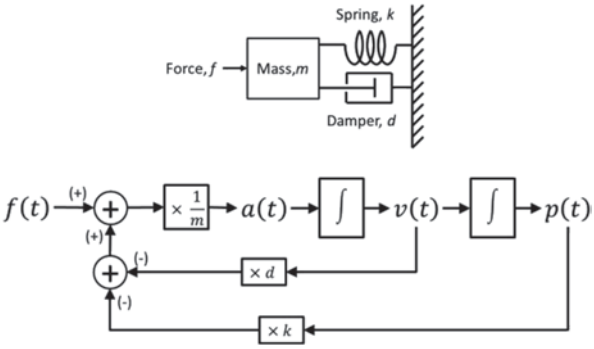
Fig. 11.3 Adding a drag term to the linear motion diagram



In this figure, acceleration is equal to force divided by mass, velocity is the integral of acceleration, and position is the integral of the velocity. The block diagram representation is very powerful because it provides an easy way to visualize the interactions and allows for simple modifications. For example, suppose you realize that air friction introduces a drag force proportional to the velocity. Adding a feedback path to the diagram is shown in Fig. 11.3.

This new model is an improvement over the original model and as our knowledge of the system grows, more paths can be added. This incremental approach to model building is very common and it is wise to include provisions for it in your modeling process. The mechanical equivalent-component model for this system is straightforward because the system itself is mechanical. The mechanical model is shown in the top of Fig. 11.4. The block diagram is shown in the bottom of Fig. 11.4. An additional, position-dependent feedback path has been added to this block diagram. Force and mass are directly equivalent in both representations. A damper supplies the velocity-dependent force (the inner feedback loop) and a spring supplies the position-dependent force (the outer feedback loop). Constants are introduced so that small, scaled force and mass in the equivalent model can represent large force and mass in the real system. There are many clever mechanical devices that have been used to represent complex relationships in block diagrams. One of the first equivalent mechanical models for solving general second-order equations, like the one in this motion example, was reported in 1875 (Thomson 1875, 1876; Irwin 2009).

Fig. 11.4 Linear motion models, mechanical equivalent *top*, block diagram *bottom*



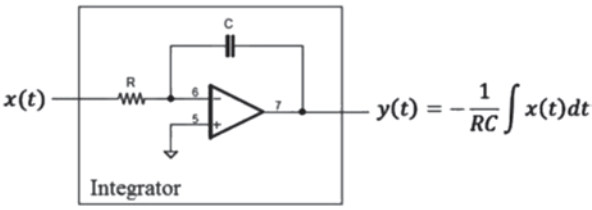


Fig. 11.5 Operational amplifier integration block

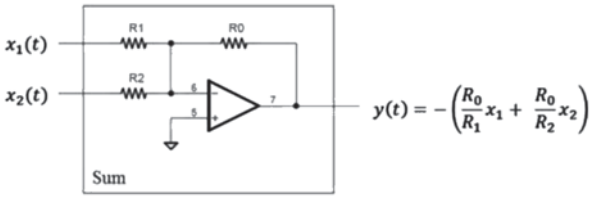


Fig. 11.6 Operational amplifier summation block

Eventually, mechanical-equivalent models gave way to electronic-equivalent models because electronic models are generally easier to set up and maintain. Before digital computers were available, analog computers were developed as general-purpose devices for solving certain types of differential equations. Operations such as addition, multiplication by a constant, and integration are implemented as separate blocks and patch-cord wires are used to connect them. Function generators supply the input and voltmeters, ammeters, and oscilloscopes are used to monitor the outputs. The same block diagram can be emulated using electronic components. Specific electronic components exhibit a differential relationship between voltage and current; however, functional blocks that use operational amplifiers (op-amp) are much easier to string together.

The workhorse block is the op-amp integrator shown in Fig. 11.5. The output of the integration block is a scaled integration of the input. The resistor and capacitor in the circuit set this scale factor. The summation block shown in Fig. 11.6 is used to tie everything together. Any number of inputs can be summed together using this circuit and the gain of each input can be separately specified. Resistors are used in each branch to set the scale for each input. Many other blocks are also available (Lundberg 2005). Connecting various blocks results in the electronic equivalent of simple motion, shown in Fig. 11.7. This circuit is capable of simulating almost any linear second-order (i.e., two integration blocks) system simply by properly selecting resistor and capacitor values that match the system equations. Circuits with dozens of blocks can be used to model higher-order systems or systems with complicated interactions. While the resulting equivalent circuit and its behavior can be very complicated, it is still assembled piece-by-piece using the same basic blocks in this example.

Issues involved in generating a suitable input, a tedious, error-prone connection process, limited dynamic range for the variables, and the advance of computer-based continuous simulation have pushed the use of equivalent-component modeling into special-purpose applications. Some high-speed or low-cost, high-volume

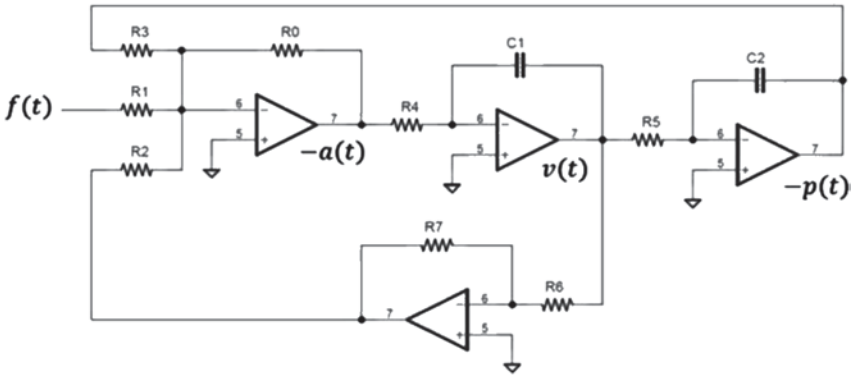


Fig. 11.7 Operational amplifier equivalent simple motion model

applications still use electronic-equivalent models. The Harvard University web site lists analog computations among their current research interests although the most recent publication listed there is 1997 (Harvard Robotics Laboratory 2013). The rapid advance, unlimited flexibility, and compelling economics of computer-based methods make the use of equivalent-component models more of an interesting historical footnote. Even though the use of equivalent-component models is falling out of favor, many of the techniques used for model development are still appropriate.

11.2.3 Computer-Based Continuous Models

11.2.3.1 Computer-Based Introduction

As a discipline, modeling often appears overwhelming because the systems we want to model are so diverse. To paraphrase the early, influential system theorist Lotfi Zadeh, nothing in the human experience is as complex and encompassing as the concept of a system (Zadeh 1962).

One way to manage this complexity is to divide the universe of systems into different categories where each can be analyzed based on the category's characteristics. At the highest level, systems have been divided into categories called hard and soft (Boulding 1956, 2006; Gigch 1991). The hard category includes physical sciences (e.g., physics, chemistry, earth sciences) and life sciences (e.g., biology, zoology, and botany) while the soft category includes behavioral sciences (e.g., psychology and sociology) and social sciences (e.g., economics, and education). In general, continuous M&S is compatible with the hard category.

The hard category can be separated further using the following criteria:

- Linear versus nonlinear
- Time invariant versus time varying
- Deterministic versus stochastic

- Continuous time versus discrete time
- Nonadaptive versus adaptive (or cognitive)
- Differential versus nondifferential
- Chaotic versus nonchaotic

Many of these categories can be further refined (Rao and Unbehauen 2006); however, a comprehensive taxonomy is not the goal. Instead, we will examine several of these criteria to understand how they support incremental model development and point out some common pitfalls.

The goal of the modeling process is to construct a model using an idealized set of variables and relationships that supports acceptably accurate solutions. The best models draw heavily from standards, academic literature, and time-tested algorithms. In the hard category, modeling has a long history and many parameterized mathematical relationships are well established (Tavernini 1996; Zadeh 1962; Zwillinger 1998; Hu et al 2010). Also true is the observation that systems in different disciplines are often governed by the same equations with different parameters. For example, second-order systems like our motion example show up in a surprisingly large number of applications. In fact, this is the basis for the equivalent-component models discussed earlier. The system-modeling process is often separated into three distinct system-identification problems: white box, gray box, and black box (Rao and Unbehauen 2006). A brief description is provided below.

- *White box*—An accurate, parameterized system model is developed from first principles and the parameter values are established based on the measured input/output behavior of the system. This is a parameter estimation problem and various techniques exist to handle linear, piecewise, and nonlinear systems of varying complexity.
- *Gray box*—Based on experience or observation, an approximate model is proposed and free model parameters are established based on the measured input/output behavior of the system. Assuming the system exhibits second-order behavior is one of these approximations. The US standard atmosphere model is one example of a gray-box model. The atmospheric model fits the data very well even though it does not strictly rely on first principles. Any sort of curve fit, including the use of trained artificial neural networks is generally classified as gray box identification.
- *Black box*—No prior model is available so the measured input/output behavior of the system is used to infer both the model and its parameters.

In all of these identification problems, it is important that the input/output data span the entire space of interest. Extrapolation (i.e., modeling areas outside the input/output space that was used for the identification) is less risky with a white box model as long as the conditions do not violate the basic modeling assumptions used to develop the model's physics-based equations. Typical assumptions are those like small deflections, low velocity, small forces, no drag or friction, uniform fields (no fringing), and so on that support linear, time-invariant model equations. Extrapolation outside of these conditions results in large errors. Extrapolation with gray- and

black-box models is dangerous because there is no basis of support outside the data used to identify the model.

11.2.3.2 Mathematical Basis for Continuous M&S

One observation about many physical systems is that their states or variables evolve or change with respect to time. In mathematical terms, derivatives can be used to represent this rate of change and the continuous models are derived using a set of differential equations. Simplifying assumptions in our model are applied with several goals in mind:

- The mathematical model must represent the system well enough to support acceptably accurate solutions.
- The mathematical model must be simple enough that it can be solved (in reasonable time and on reasonable hardware).

The degree of acceptability or reasonableness in these goals is always a difficult thing to pin down. Model results should agree with physical observations but the allowable error depends on how the model results are being used. For example, in a global-annihilation video game, it might be perfectly acceptable to calculate the trajectory of intercontinental ballistic missiles assuming that the earth is round, that gravity is uniform, that coriolis forces do not exist, and that the atmosphere adds no drag. If instead you are trying to calculate the trajectory of a supply ship docking with the international space station, all of these relationships and more need to be included in the model equations. It is instructive to look at several simple models and see how they can be augmented to more closely match reality.

In the Malthusian model of population growth, the time rate of change in the number of members in a population is proportional to the size of the population. The differential equation for Malthusian growth is (Wikipedia Malthusian 2013)

$$\dot{p}(t) = rp(t) \tag{11.1}$$

where dot represents a derivative with respect to time, p is the number of members in the population, and r is the Malthusian constant that balances the number of births and deaths. The Laplace domain transfer function can be expressed as[1]

$$H(s) = \frac{1}{s - r} \tag{11.2}$$

and the solution to this simple first-order equation is,

$$p(t) = p(0)\mathrm{e}^{rt} \tag{11.3}$$

[1] For a linear, time-invariant differential equation, the transfer function can be calculated using $H(s) = C(sI - A)^{-1}B$. Here, $A = r$, $B = 1$, $C = 1$, and $I = [1]$.

The solution is consistent with the often-repeated statement about exponential growth. If constant r is greater than zero, births outnumber deaths, the system pole is in the right-half plane, and in the limit, the population will approach infinity. If constant r is less than zero, deaths outnumber births, the system pole is in the left-half plane, and in the limit, the population will approach zero. This model is reasonably accurate when the population is living within its resources. As the population approaches resource limits, the growth rate slows. To accommodate the change, either r needs to be a function of population or the structure of the equation needs to change. The Verhulst model changes the structure to include a term that slows the growth rate as the population grows (Wikipedia Logistic 2013). The Verhulst equation is

$$\dot{p}(t) = rp(t)\left[1 - \frac{p(t)}{K}\right] \tag{11.4}$$

where the constant K is called the carrying capacity. The system is no longer linear so a transfer function description is not meaningful. The solution to this nonlinear differential equation is a form of the well-known function called the logistic function given by,

$$p(t) = \frac{Kp(0)e^{rt}}{K + p(0)(e^{rt} - 1)} \tag{11.5}$$

This function rises exponentially near $p(0)$ but then rolls over to approach an asymptote of $p(t) = K$ as t approaches infinity. A parameterized plot of the resource-limited population versus time described by the logistic equation is shown in Fig. 11.8.[2]

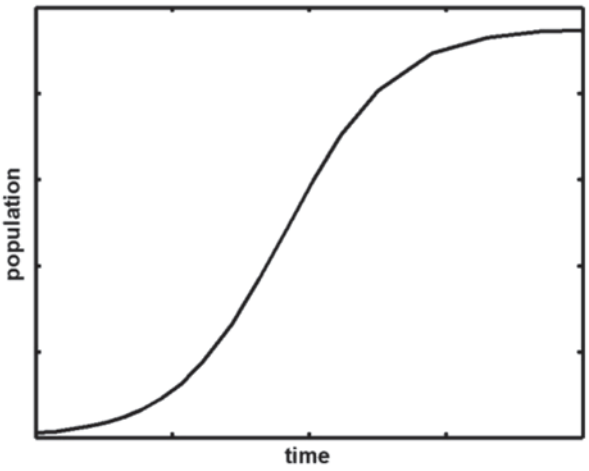


Fig. 11.8 Plot of the logistic function, solution to the Verhulst population model

[2] MATLAB's ODE23 was used to generate the data in the plot.

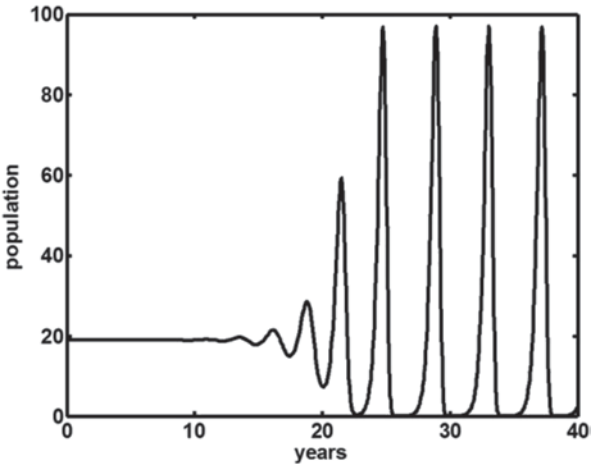


Fig. 11.9 Plot of the regarded logistic function, single-population oscillations

Units for time and population are omitted because they depend on specific populations. The important feature is the shape.

Almost any integration method or differential equation solver can handle this kind of simple differential equation.

Now suppose the density-dependent term has a delayed effect. The idea here is that young members of the population need to reach a certain age before their effect is seen. This is the so-called retarded logistic equation written as,

$$\dot{p}(t) = rp(t)\left[1 - \frac{p(t-\tau)}{K}\right] \tag{11.6}$$

where τ is a delay. In a linear system, a delay in the feedback path tends to degrade system stability and we expect the same thing to happen here. With suitable parameters (Tavernini 1996) ($r=3.5$, $K=19$, $\tau=0.74$, and $p(0)=19$) a single population will advance and crash with approximately a 4-year period. A plot of population versus time is shown in Fig. 11.9.[3]

Eventually, the population dynamics settle into a stable limit cycle where the size of the population rises and falls in the same way, cycle after cycle. This can be seen in the phase diagram where population is plotted against the population change, Fig. 11.10. The initial population point is in the middle of this curve at (20,0). As time goes by, the population circles this point as it expands and contracts. The final oscillating solution is represented by the outside steady-state curve; the limit cycle. Once the solution spirals out to this curve, it follows that path indefinitely.

The time delay introduces interesting dynamics into the solution and it introduces interesting implementation issues. In general, delays involve keeping a history of solutions and interpolating values when time steps do not exactly match the delay

[3] MATLAB's dde23 function was used to generate the data in the plot (Shampine 2000; Shampine and Thompson 2001).

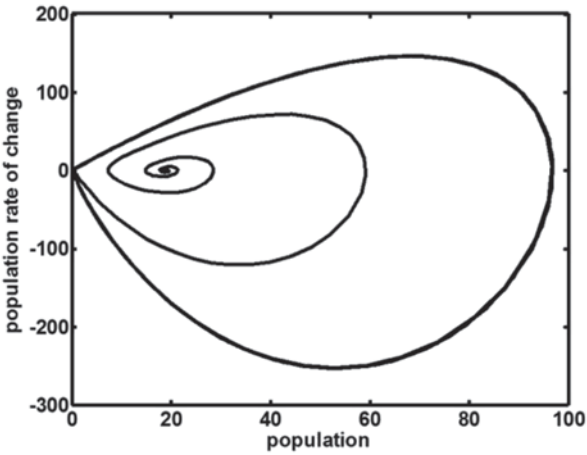


Fig. 11.10 Phase plane plot of the regarded logistic function, single-population oscillations

time. To keep everything in synch, the interpolation function and the integration function must be compatible.

Discontinuities in low-order derivatives also require special considerations. The typical treatment limits the integration time step near a discontinuity. Because of this, delay differential equations (DDE) with general delays are considerably more difficult. Fortunately, constant-delay problems represent a large class of DDEs (Shampine 2000).

Other situations that can lead to limit cycles involve coupled equations. Using population as an example, the Lotka–Volterra equation is a simple representation of the way that populations of predator versus prey interact over time. The second-order, nonlinear Lotka–Volterra equations are written as,

$$\begin{aligned} \dot{x}(t) &= ax(t) - bx(t)y(t) \\ \dot{y}(t) &= -cy(t) + dx(t)y(t) \end{aligned} \tag{11.7}$$

where x is the number of prey (e.g., rabbits), y is the number of predators (e.g., foxes), and the constants (a, b, c, d) describe the interaction between the two species. It turns out that this form of second order equation has been used to model a wide variety of problems that include ecological systems, disease, chemical reactions, and economics, among others.

The equilibrium solution can be found by setting the derivatives to zero and solving for x and y. Doing this yields,

$$\begin{aligned} x_e &= \frac{c}{d} \\ y_e &= \frac{a}{b} \end{aligned} \tag{11.8}$$

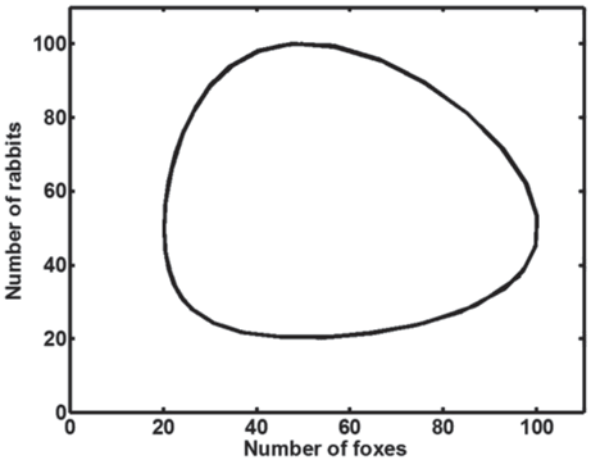


Fig. 11.11 Phase plane plot of Lotka–Volterra equations

The solution to the Lotka–Voltera equations is a family of limit cycles that orbit this equilibrium point (x_e, y_e) and pass through the initial conditions. An example plot ($a=0.5$, $b=0.01$, $c=0.5$, $d=0.01$, $x(0)=50$, $y(0)=100$) is shown in Fig. 11.11.[4]

Once the simple solution is understood, incremental changes can be applied. For example, suppose that foxes (i.e., predators) are periodically removed from the system. The equations might look like,

$$\begin{aligned} \dot{x}(t) &= ax(t) - bx(t)y(t) \\ \dot{y}(t) &= -cy(t) + dx(t)y(t) - h(t) \end{aligned} \tag{11.9}$$

where $h(t)$ is a term that represents fox removal. This leads to an imbalance and a continuously outward spiraling solution. Now suppose that availability of resources limits the rabbit population. Now the equations might look like,

$$\begin{aligned} \dot{x}(t) &= ax(t)\left[1 - \frac{x(t)}{K}\right] - bx(t)y(t) \\ \dot{y}(t) &= -cy(t) + dx(t)y(t) - h(t) \end{aligned} \tag{11.10}$$

where constant K is the carrying capacity for the rabbits. An example plot ($a=0.5$, $b=0.01$, $c=0.5$, $d=0.01$, $h(t)=1$, $K=200$, $x(0)=50$, $y(0)=100$) is shown in Fig. 11.12.[5] Even though the foxes are being removed, the resource limits keep the system stable. For this particular choice of parameters, the solution settles into a particular solution for the number of foxes and rabbits.

There is any number of other variations that could be added to the model equations. For example, suppose there are two prey species described by a set of three

[4] MATLAB's ode45 function was used to generate the data in the plot.

[5] MATLAB's ode45 function was used to generate the data in the plot.

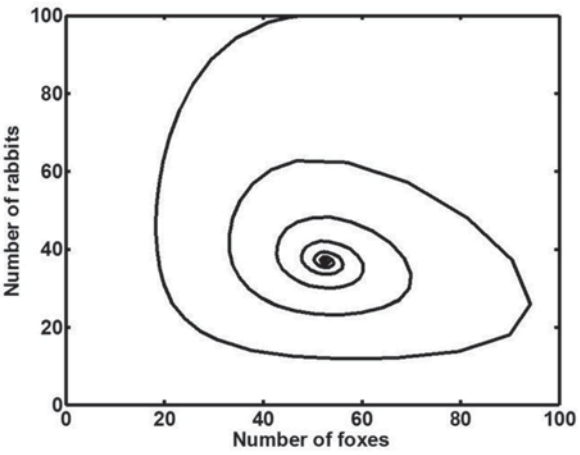


Fig. 11.12 Phase plane plot of predator–prey equations with hunting and resources

coupled equations. Here, you could also include some level of resource competition between prey species. In another example, suppose that foxes do not become hunters until they reach a certain age. This adds a delay term and changes the nature of the problem into a delay differential equation. There are two interesting things to observe from the progression of these population models. The first is that extending a model to fit your particular system is often a process of incrementally adding terms until the desired effects or accuracy are achieved. The second is that many dynamic equations have already been extensively studied, giving you ample opportunity to test your simulation with known parameters before tailoring it to your particular conditions.

11.2.3.3 Hybrid Systems

There are many real-world systems that include regions of continuous dynamics connected through instances of discrete behavior. The continuous dynamics are described by differential equations while the discrete behavior might be described by a difference equation, a control graph, or a discrete event. The thermostat in your home is a simple example. Changes in room temperature can be modeled as a dynamic system but the control of the air conditioner is discrete: on or off. The continuous and discrete dynamics interact so that changes to the continuous subsystem influence the discrete subsystem and vice versa. A bouncing ball, a bang–bang controller, a digital control system, coulomb friction, and gear backlash are all examples of this behavior.

A system that includes a combination of both continuous and discrete dynamics is generally referred to as a hybrid system (or sometimes as a hybrid dynamic system). Different disciplines tend to apply the hybrid system name to particular subcategories. Computer engineering may refer to embedded systems, that is, a digital system interacting with the real world, as hybrid. The M&S community may refer to physical systems that operate in different modes with discrete, instantaneous transi-

tions as hybrid. The control systems community may refer to hierarchical systems with a discrete decision layer and a continuous implementation layer as hybrid (van der Schaft and Schumacher 2000). Different approaches may be required in these different subclasses so it is important to be aware of the context.

M&S approaches to hybrid dynamic systems have been classified as implicit or explicit (Fishwick 2007). The implicit approach generally results in differential algebraic equations that are better suited for analysis rather than simulation. The reason for this is the use of algebraic loops that are difficult to code reliably. One example of the implicit approach can be used to model a sharp transition point by an extremely fast exponential function. The advantage is continuous derivatives at the transition point. Some texts call this the smoothing method (van der Schaft and Schumacher 2000).

The explicit approach uses ordinary differential equations tied together by a finite state machine. A state transition diagram is used to represent the transition between different solution modes. The explicit approach has been formalized as a hybrid automaton (also known as a dynamic automaton) and many good examples have been described and studied (Tavernini 1996; Fishwick 2007). Application areas include friction, hysteresis, backlash, bang–bang control, collisions, electronic switching, and digital controllers, among others. In a slightly different approach, a generalized discrete event specification (GDES) methodology has also been established to explicitly handle hybrid dynamic systems (Giambiasi et al. 2000). GDES allows hybrid dynamic systems to be defined so they are compatible with distributed simulation environments like the Simulation Interoperability Standards Organization's (SISO) high-level architecture (HLA; see Chap. 20). It appears that piecewise linear inputs may be a limitation in GDES; however, inputs to a continuous model are a problem area for any implementation.

An example of a dynamic automaton representation of Coulomb friction will help introduce the hybrid system concept. A representative plot of friction versus velocity is shown in Fig. 11.13 where f_s is the static force and f_d is the dynamic

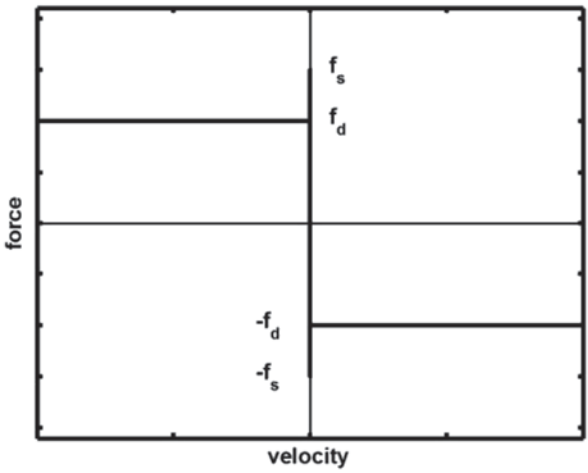


Fig. 11.13 Coulomb friction

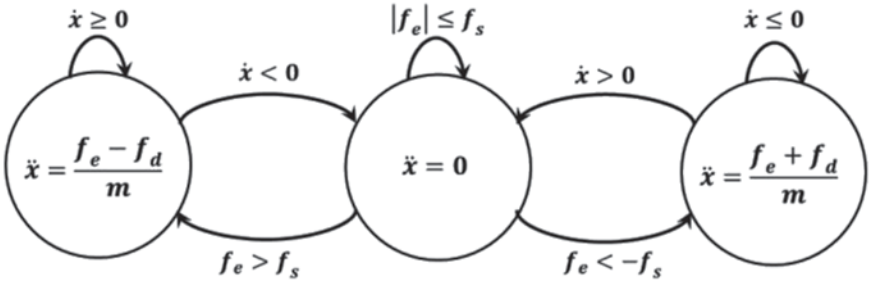


Fig. 11.14 Coulomb friction dynamic automaton

force. The static force represents the so-called condition of stiction that results in a slightly higher frictional force when an object is at rest (static) compared to when it is moving (dynamic). When velocity is zero, the static frictional force opposes the applied force. When the applied force is large enough to overcome static friction, the object begins to move with a velocity determined by Newton's second law, $f = m \times a$, using the resultant force that includes both the applied force and friction. The conditional equation for this static–dynamic friction behavior is,

$$\begin{cases} \ddot{x} = \dfrac{f_e - f_d}{m}, & \dot{x} > 0 \\ \ddot{x} = 0, & \dot{x} = 0 \text{ and } |f_e| < f_s \\ \ddot{x} = \dfrac{f_e + f_d}{m}, & \dot{x} < 0 \end{cases} \tag{11.11}$$

where $\ddot{x}$ is the acceleration, $\dot{x}$ is the velocity, f_e is the externally applied force, f_s is the static friction force, f_d is the dynamic friction force, and m is the mass. Different regions in this equation are depicted in the diagram of the dynamic automaton shown in Fig. 11.14. This dynamic automaton is similar to a finite-state automaton except that the state transitions depend both on the input and on the system variables $(x, \dot{x}, \ddot{x} \cdots)$. In comparison, a finite-state automaton depends only on the input.

Values are continuous across the transition. While new equations are used in each system state, initial conditions depend on values from the previous state. One way to think about this is to allow each finite-state transition to move the system onto a different plane. Each plane evolves according to its own dynamics but the transition from plane to plane occurs at switch points (see Fishwick 2007, pp. 15−16).

An example that follows directly from (Tavernini 1996) will demonstrate how this particular dynamic automaton can be coded. The mechanical diagram is shown in Fig. 11.15. A mass rests on a flat surface and a spring is used to connect the mass to the external stimulus. A displacement in y causes the spring to stretch or contract, which applies a force to the mass. The equation for spring force is a quadratic function of the displacement given by,

$$f = K_1(y - x) + sign(y - x)K_2(y - x)^2 \tag{11.12}$$

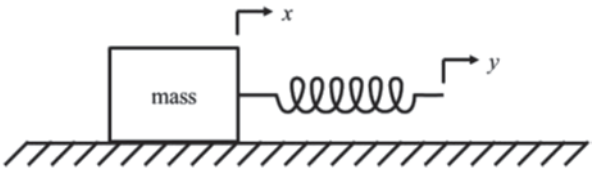


Fig. 11.15 Friction example, mechanical drawing

```
1  function friction = frictionFunction(velocity)
2    if velocity == 0
2      friction = cStatic*mass*gravity;
3    else
4      friction = -
sign(velocity)*cDynamic*mass*gravity;
5    end
6  end
```

Fig. 11.16 Frictional force calculation

where K_1 and K_2 are the spring's force parameters. In Fig. 11.15, x and y have an initial offset but in the software implementation, the reference point for the mass' displacement can be moved to coincide with the spring's initial position. Doing this makes the spring's displacement $x–y$. The sign() function is used because both terms apply force in the same direction, positive when the spring is stretched and negative when the spring is compressed.

Once the spring force overcomes static friction, mass motion is measured as a displacement in x along with higher-order derivatives. The code convention used below is MATLAB syntax, but it should easy for any programmer to follow the logic. The first calculation is the friction force coded in Fig. 11.16. If the mass is not moving, the coefficient of static friction, *cStatic,* is used. If the mass is moving, the coefficient of dynamic friction, *cDynamic,* is used and the direction opposes the velocity. Multiplying the coefficient by mass and gravity is required since that is how coefficients of friction are defined. In Fig. 11.16 we have assumed that *cStatic, cDynamic, mass,* and *gravity* are available constants so that *frictionFunction* only depends on *velocity* as an input argument.

The next major function is the implementation of Newton's law coded within the structure of the dynamic automaton. The major structure for the automaton is shown in Fig. 11.17. In the function prototype on line 1, x represents the vector of elements $(x, \dot{x})$ and dxdt represents the vector of elements $(\dot{x}, \ddot{x})$. The first call in line 2 calculates the spring force based on time and mass position, $x(1)$. After that, the *automatonState* value is inspected and one of three cases is executed based on the result. The automaton can be in one of three states, at rest ("rest"), moving in the positive direction ("pos"), or moving in the negative direction ("neg"). Code for each case, switches the *automatonState* variable and calculates the total force. The body for each case condition will be presented shortly. Line 11 implements a linear equation for Newton's law. If you are not familiar with MATLAB, the vector syntax might not be easy to decipher. In mathematical terms, the equivalent linear state equation in Line 11 is,

Fig. 11.17 Structure for the implantation of the friction dynamic automaton

```
1  function dxdt = automatonFunction(t, x)
2    forceSpring = springFun(t, x(1));
3    switch automatonState
4      case 'rest'
5        ...
6      case 'pos'
7        ...
8      case 'neg'
9        ...
10   end
11   dxdt = [0 1; 0 0] * x + [0; 1/mass] * f;
12 end
```

$$\begin{bmatrix} \dot{x}(t) \\ \ddot{x}(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x(t) \\ \dot{x}(t) \end{bmatrix} + \begin{bmatrix} 0 \\ \dfrac{1}{\text{mass}} \end{bmatrix} f(t) \tag{11.13}$$

Next, look at each case in Fig. 11.17. The directed arrows in Fig. 11.14 are used to establish the various conditions for entering and exiting each state. The "rest" case is shown in Fig. 11.18. There are three exit arrows in Fig. 11.14 and three conditions inside the "rest" case. The first on line 2 is used when the spring force is larger than the static friction (i.e., *frictionFunction* with an input velocity of zero will return the static friction force). For this condition, the mass begins to move in the positive direction ("pos"), the velocity is now positive (*eps* is a very small positive number, 2.2E-16), and the force available to accelerate the mass is the spring force minus the frictional force (*frictionFunction* will return a negative value for a positive input velocity). The second condition on line 6 is used when the spring force is more negative than the static friction. Variables for this condition are similar to the first case except in the opposite direction. The third condition on line 10 is used when the spring force is not yet large enough to overcome static friction. In this condition, the velocity is zero and there is no force available to accelerate the mass.

The "pos" case is shown in Fig. 11.19. There are two exit arrows in Fig. 11.14 and two conditions inside the "pos" case. The first on line 2 is used when the velocity changes from positive to negative. The automaton cannot switch directly to the negative direction but rather, has to stop and enter the "rest" state first. When it is in the "rest" state, the velocity has to be zero and the force available to accelerate the mass is also zero. The second condition on line 6 is used when the mass is still moving in the positive direction. The force available to accelerate the mass is a combination of spring force plus friction.

The "neg" case is shown in Fig. 11.20. It is almost identical to the "pos" case except that the condition on line 2 looks for a change from negative velocity to positive. Other than that, the "neg" behavior is the same as "pos"

Fig. 11.18 "Rest" case for the friction example

```
case 'rest'
  if forceSpring > frictionFunction(0)
    automatonState = 'pos';
    x(2) = eps;
    f = forceSpring + frictionFun              c-
tion(eps);
  elseif forceS <         - frictionFunction(0)
    au    tomatonState = 'neg';
    x(2) =       - eps;
    f = forceSpring + frictionFunction(                -
eps);
  else
    automatonState = 'rest';
    x(2) = 0;
    f = 0;
  end
```

Fig. 11.19 "pos" case for the friction example

```
case 'pos'
  if x(2) <= 0
    automatonState = 'rest';
    x(2) = 0;
    f = 0;
  else
    f = forceSpring + frictionFun(x(2));
  end
```

Fig. 11.20 "neg" case for the friction example

```
case 'neg'
  if x(2) >= 0
    automatonState = 'rest';
    x(2) = 0;
    f = 0;
  else
    f = forceSpring + frictionFun(x(2));
  end
```

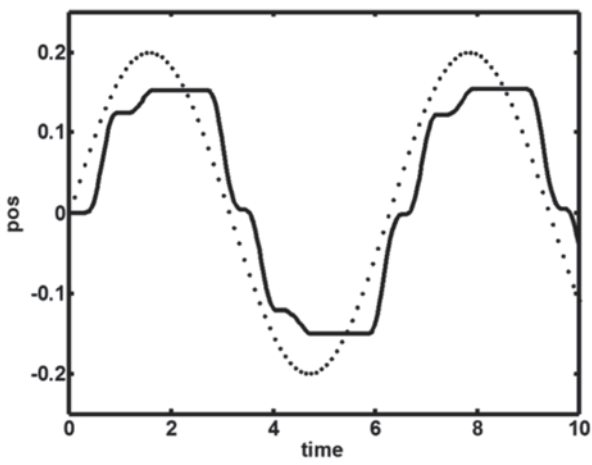


Fig. 11.21 Plot of input to the spring (*dotted*) and output mass motion with friction (*solid*)

Finally, define the input y to be a sinusoid with amplitude a and frequency ω coded as,

$$y = a*\sin(\omega*t);$$

Now, a reference to the automaton is used to call the solver. Again in MATLAB syntax, the solver call is coded as,

$$[t,\ x] = \text{ode23}(@\text{automatonFunction},[010],[0;\ 0]);$$

MATLAB's *ode23* differential equation solver uses a Runge–Kutta technique that is suitable for low-order equations that can be characterized as nonstiff. A description of Runge–Kutta is provided in the next subsection. The solver exhibits relatively large errors but is adequate for this problem. The first argument in the *ode23* call is a reference to a function that defines the system of differential equations. In this example, that function is the friction automaton function. The second argument is the time span. Here, the time span is from zero to 10 s. The third argument is the initial condition vector. Here, both position and velocity are initialized to zero. The values returned by the function are a vector of time values and a corresponding set of $(x, \dot{x})$ pairs. The resulting plots of x (mass position, solid line) and y (input drive position, dotted line) are shown in Fig. 11.21. System parameters are set up to achieve a jerky, start–stop sort of motion for a smoothly oscillating input. That is exactly what the plot shows.

11.2.3.4 Integration

Computer-based models use special mathematical techniques to support M&S of continuous time systems using an inherently discrete time computer. When analog

computing gave way to digital computing, the first type of continuous-simulation software mimicked the use of the summing blocks, multiplying blocks, and integration blocks because these were already familiar in the M&S community. In those early days, a graphical interface and a mouse were not available. Consequently, the models were difficult to configure and visualize. Without good graphics, engineers and scientists found it much easier to specify the problem using equations instead of block diagrams. As a result, the second generation of continuous simulation software emphasized equations over diagrams. Today, graphical interfaces are normal and continuous simulation environments often use a combination of block diagrams and equations to specify the model. There are also continuous-simulation libraries (both open source and commercial) for established languages like formula translator (FORTRAN), C++, MATLAB, or Python as well as special purpose languages, like advanced computer simulation language (ACSL)[6] and partial differential equation language (PEDL),[7] specifically tailored for continuous M&S. The National Institute of Standards and Technology (NIST) maintains a long-running project called the Guide to Available Mathematical Software (GAMS; NIST 2013).[8] The purpose of GAMS is to produce a taxonomy of mathematical software organized by the type of problem that it solves. A portion of this indexed repository is available to the public. Oakridge National Laboratory, University of Tennessee, Bell Labs, and "collegues worldwide" a publically indexed repository at www.netlib.org. The repository represents 9000 problem-solving modules from more than 100 packages. The library indexes almost every significant numerical package and serves as a great starting point for any search.

Generally, these numerical libraries wrap numerical complexity into differential equation solvers. In these solvers, there is a lot behind the curtain but arguably, the most important component is a numerical integrator. The solution method is integration but the implementation varies widely depending on requirements that include problem type, accuracy, and run-time efficiency, among others. What follows is a rather brief look at numerical integration along with guidelines that can be used to select an appropriate solver.

Integrating a function is different from integrating a set of differential equations. For a function, we can calculate the value at any point and various numerical techniques solve the integration problem by judiciously selecting a curve that passes through selected points on the function. The shape of this curve is selected so that it both closely follows the original function and has a known, closed-form answer. Typically, we select equally spaced points and use a polynomial. The familiar trapezoidal rule uses two points and a straight-line (i.e., first order). Simpson's rule uses three points and a second-order curve. Using more points can be generalized and is known as Newton–Cotes. Newton–Cotes comes in two varieties: closed and open. In closed Newton–Cotes, the set of points exactly spans the integrated area. In open Newton–Cotes, points outside the area of integration are used to calculate coef-

[6] ACSL—Advanced computer simulation language

[7] PEDL—Partial differential equation language

[8] Identification of products in GAMS does not imply recommendation or endorsement by NIST.

ficients. Other techniques substitute different methods of selecting the new curve. Gaussian quadrature[9] uses a polynomial but the points do not have to be equally spaced. Orthogonal polynomials like Legendre, Laguerre, Chebyshev, and Hermite have all been used in specific application areas. For most problems, Gaussian quadrature is generally preferred and adaptive step-size selection can be used to speed up the calculations (Teukolsky 2007).

Solving a set of differential equations is different from integrating a function because we are using integration to help approximate the function itself. This means that we cannot calculate values at arbitrary points and approximate the solution using a polynomial. Instead, we can calculate values at some specific or convenient points and estimate the forward solution as we go. Basically, it comes down to a difference between interpolation and extrapolation. For example, we know the starting point because initial conditions must be specified along with the equations.[10] Given the equations and the initial point, the basic idea for estimating the system is to rewrite the equations in terms of finite steps. The initial conditions are used to extrapolate the solution to the next step. This yields algebraic equations that can be solved at every step. In the limit, as the step size approaches zero, a good approximation results. Euler's method is a first-order method that follows this approach; however, aside from demonstrating the concept, Euler's method is not useful as a serious numerical technique. Runge–Kutta and predictor–corrector methods are the most common general methods.[11]

Runge–Kutta methods first sample the solution at steps smaller than the overall step size. These samples are then included in the estimate when the entire step is extrapolated. The sub-step samples are used to match the coefficients of a higher-order Taylor series expansion. The fourth-order Runge–Kutta method is often referred to as RK4 or simply as the Runge–Kutta method. RK4 uses four samples to calculate a solution for the value at t_{n+1}one sample at t_n, two samples at $t_{n+1/2}$, and finally one sample at t_{n+1}. These sample values are then used in a polynomial equation that extrapolates the system from t_n to t_{n+1}. In most applications, Runge–Kutta will be able to solve the system. This makes Runge–Kutta the method of choice for problems where computational efficiency is not a big concern. Higher-order Runge–Kutta methods exist but it is more common to use a predictor–corrector method instead of, for example, RK6.

If we knew the trajectory of the system in advance, integrating the system equations would be the same as integrating a function. The predictor–corrector methods use a prediction phase to make the problem appear as if we are integrating a function. The corrector phase uses this predicted value to refine the answer. Predictor–

[9] Numerical integration is synonymous with the term "quadrature."

[10] There are also boundary-value problems that require different techniques like shooting and relaxation, among others.Partial differential equations and integro-differential equations require all-together different techniques.The initial value problem helps form a good foundation for investigating other problems.

[11] Bulirsch–Stoer has been predicted to supplant predictor–corrector methods for most applications (Press 2007); however, the use of predictor–corrector methods is still common.

corrector methods use one polynomial to extrapolate the solution to the next point and another polynomial to add the correction. Wikipedia mentions that this correction step can be run twice or even iteratively (Wikipedia Predictor–corrector 2013). Press recommends using a smaller step size rather than repeating the correction (Teukolsky 2007). Libraries seem to be divided and often include both approaches. One caveat to iterative correction is the run-time uncertainty involved in repeating a correction until an error bound is achieved. Initialization can also be a problem since the predictor relies on previous information to extrapolate a solution. Packaged library functions will include a standard initialization process. For reasonably smooth problems, predictor–corrector methods are more efficient compared to Runge–Kutta. The most popular implementation for predictor–corrector methods is Adams–Bashforth–Moulton. Adams–Bashforth is the predictor and Moulton is the corrector. Polynomial coefficients for the Adams–Bashforth–Moulton method are well known (Carnahan et al. 1969; Teukolsky 2007).

The first rule in numerical methods is "use prepared software packages whenever possible" (Zwillinger 1998). The amount of open-source software that is available and the ease at which it can be found (NIST 2013) makes this rule easy to follow as long as you can figure out which piece of software to use. The discussion up to this point should help in that regard. Table 11.1 provides additional information that can be used as a rough guide to further refine that choice.

It is hard to talk about approximation without also talking about errors. There are two types of errors involved: truncation and round off. Truncation errors describe the mismatch between the approximation and the actual solution. For example, second-order polynomials have third-order errors. That is, a second-order polynomial has the chance to match the shape of a second-order solution without any error. If the solution is actually higher order, the largest error term using a second-order polynomial will be proportional to the step size cubed. Because of this, smaller step sizes produce smaller truncation errors.

Round-off errors occur due to the inability of fixed-length digital registers to hold arbitrary values. Double-precision floating point is usually adequate. It is also a good idea to try to balance truncation and round off. Using a higher-order method or a smaller step size (to reduce truncation error) does not improve accuracy if the major source of error is round off. Similarly, increasing the precision of variables (to reduce round-off error) does not improve accuracy if the major source of error is truncation.

Exceptions to the statement that double precision is usually adequate can occur when the system trajectory includes both extremely large and extremely small values that interact. This is basically what happens for so-called stiff systems. One part of the solution decays much more rapidly than the other resulting in a many- orders-of-magnitude difference between the system trajectories. System eigenvalues can be used to calculate a stiffness ratio; however, the more typical scenario is trial and error. Start with a general numerical technique (maybe even low order) and a reasonable step size. Progress to higher order and more complexity until the solution converges. Another idea is to start with a simplified version of the system equations and incrementally add sophistication to both the equations and numerical methods.

Table 11.1 Differential equation solver guide

Method	Application	Recommendation
Euler	Teaching	The Euler method is not a serious tool for numerical methods. It is easy to code and can be used as an instructional aide for an introductory numerical methods course
2nd order Runge–Kutta (RK2)	Smooth	Poor error performance relegates RK2 to problems where only rough estimates are required but computational efficiency is important. RK2 is a good coding problem for an introductory course in numerical methods
3rd order Runge–Kutta (RK3)	General	Better error performance than RK2 but RK4 would be preferred unless computational efficiency is important. A modification called the Bogacki–Shampine method supports adaptive steps size and is used by MATLAB's ode23 function
4th order Runge–Kutta (RK4)	General	RK4 is the first method to try. The MATLAB ode45 function uses a variant attributed to J. Dormand and P. Prince. (Shampine and Reichelt 1997) RK4 is also a good choice as the integration method for delay-differential-equation solvers
Adams–Bashforth–Moulton (ABM)	General	ABM4 is preferred over RK4 when low error and computational efficiency are desired. Higher-order ABM generally lowers errors even further. The MATLAB ode113 is an ABM method up to 12th order (Shampine and Reichelt 1997)
Klopfenstein–Shampine	Stiff	Try this method if RK4 fails to converge or runs too slowly. The MATLAB ode15s uses this method (Shampine and Reichelt 1997)
Rosenbrock (Kaps 1985)	Stiff	This method has advantages when one or more of the system eigenvalues are near the jω axis. The MATLAB ode23s uses this method (Shampine and Reichelt 1997)
Bulirsch–Stoer	General	This method has been identified as possibly a better alternative to ABM methods and should be suitable when high accuracy is desired (Teukolsky et al 2007)

Sometimes, this makes it easier to spot the change that causes the method to veer off course. Special techniques along with scaling can also be used to help deal with these situations and are generally encapsulated in library functions.

11.3 Conclusions

This has been a brief tour of some of the aspects involved in continuous-time M&S. This tour was designed to both provide a good foundation for continuous time M&S and provide a contrast with discrete time M&S. The major differences between

continuous time and discrete time models are due to the underlying mathematical relationships. Continuous time models rely on differential equations and different methods for continuous time simulation try to solve these equations in different ways. Scale models attempt to physically recreate the conditions where the differential equations can exist. Equivalent component models try to emulate these differential equations with a mechanical or electrical replacement. Computer models try to solve the differential equations by applying numerical techniques that allow a discrete-time computer to estimate the real-time relationships. The most important numerical techniques involve integration and we briefly looked methods for integrating functions and integrating system equations. Integrating functions is easier since the function values are known, or at least can be calculated, at any point. Integrating the system equations is more difficult because we need to extrapolate the solution to the next sample point. Various methods are available and the nature of the problem and its expected solution, along with the computational speed required, need to be considered when trying to decide on the best method.

References

Alan Oppenheim and Ronald Schafer (1975) Digital signal processing, Prentice-Hall, New Jersey. ISBN 0132146355

Box GEP (1976) Science and statistics. J Am Statist Assoc 71(356):791–799

Box GEP, Draper NR (1987) Empirical model-building and response surfaces, Wiley, New York, p 424 ISBN 0471810339

Boulding K (1956) General systems theory, the skeleton of science. Manage Sci 2(3):197–208

Carnahan B, Luther HA, James O (1969) Wilkes, applied numerical methods. Wiley, New York

Fishwick PA (2007) Handbook of dynamic system modeling. CRC, Boca Raton. ISBN 1420010859

Giambiasi N, Escude B, Ghosh S (2000) GDEVS a generalized discrete event specification for accurate modeling of dynamic systems, trans. SCS 17(3):120–134

Gigch van J (1991) System design modeling and metamodeling. Plenum, New York. ISBN 0306437406

Harvard Robotics Laboratory, Analog Computation (2013) http://hrl.harvard.edu/analog/. Accessed 1 Oct 2013

Hu X, Jonsson Ulf, Wahlberg Bo, Ghosh BK (eds) (2010) Three decades of progress in control science. Springer, New York. ISBN 9783642112775

International Towing Tank Conference (2012) Recommended procedures and guidelines, speed and power trials, part 1 preparation and conduct, rev. 1, 2012, http://ittc.sname.org/7%205-04-01-01%201%20Preparation%20and%20conduct%20of%20speed%20power%20trials.pdf. Accessed 10 Sept 2013

Irwin W (2009) The differential analyzer explained. Auckland Meccano Guild. http://amg.nzfmm.co.nz/differential_analyser_explained.html. Accessed 1 Oct 2013

Kailath T (1980) Linear systems. Prentice-Hall, New Jersey. ISBN 0135369614

Kaps P, Poon SW, Bui TD (1985) Rosenbrock methods for stiff ODEs. Computing 34:17–40

Lee B (1994) Lazzarini's lucky approximation of π. Math Mag 67(2):83–91 http://www.jstor.org/stable/2690682. Accessed 11 Oct 2013

Lundberg KH (2005) History of analog computing. IEEE Control Syst Mag 25(3):22–28

Metropolis N (1987) The beginning of the Monte Carlo method, Los Alamos Science (1987 Special Issue dedicated to Stanisław Ulam): 125–130, 1987, http://library.lanl.gov/la-pubs/00326866.pdf

National Institute of Standards and Technology, GAMS Background (2013) http://gams.nist.gov/Background.html. Accessed 20 Sept 2013.
National Oceanic and Atmospheric Administration, U.S. Standard Atmosphere (1976) U.S. Government printing office, Washington, DC
Press W, Teukolsky S, Vetterling W, Flannery B (2007) The art of scientific computing, 3rd edn. Cambridge University Press, Cambridge. ISBN 9780521880688
Przemyslaw B, Shampine LF (1989) A 3(2) pair of Runge–Kutta formulas. Appl Math Lett 2(4):321–325
Rao GP, Unbehauen H (2006) Identification of continuous-time systems. IEE Proc-Control Theory Appl 153(2):185–220
Robinson MC (1992) Rivers in miniature: the Mississippi basin model. In: Barry F (ed) Builders and fighters: U.S. Army Engineers in World War II. Fort Belvoir: Office of History
Shampine LF (2000) Solving delay differential equations with dde23. http://www.math.ntnu.no/~hek/MatMod2009/Modelleringsseminar/ShampineThompson_DelayDifferentialEquations23_Tutorial.pdf. Accessed 7 Oct 2013
Shampine LF, Reichelt MW (1997) The MATLAB ODE Suite. SIAM J Scientific Computing 18–1
Shampine LF, Thompson S (2001) Solving DDEs in MATLAB. Appl Numer Math 37:441–458
Tavernini L (1996) Continuous-time modeling and simulation. Overseas Publishers association, ISBN 2-88449-224-0
Thomson J (1875) On an integrating machine having a new kinematic principle. Proc R Soc Lond 24(164–170):262–265
Thomson W (1876) Mechanical integration of the linear differential equations of the second order with variable coefficients. Proc R Soc 24:269–268
U.S. Army Corps of Engineers (2013) Wikimedia commons. http://chl.erdc.usace.army.mil/chl.aspx?p=FullSizeImage&a=ImageFile_ID;2094. Accessed 30 Sept 2013
van der Schaft AJ, Schumacher JM (2000) An introduction to hybrid dynamic systems. Springer, New York. ISBN 1852332336
Wikipedia contributors (2013) Ames Research Center. Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Ames_Research_Centeroldid=572136309. Accessed 10 Sept 2013.
Wikipedia contributors (2013) Buffon's needle. Wikipedia, the free encyclopedia, http://en.wikipedia.org/w/index.php?title=Buffon%27s_needleoldid=572902363. Accessed 11 Oct 2013
Wikipedia contributors (2013) Lavarand. Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Lavarandoldid=572230424. Accessed 10 Oct 2013
Wikipedia contributors (2013) Logistic function. Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Logistic_functionoldid=564137153. Accessed 3 Oct 2013
Wikipedia contributors (2013) Lotka–Volterra equation. Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Lotka%E2%80%93Volterra_equationoldid=575582927. Accessed 7 Oct 2013
Wikipedia contributors (2013) Malthusian growth model. Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Malthusian_growth_modeloldid=574091511. Accessed 3 Oct 2013
Wikipedia contributors (2013) Predictor–corrector method. Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Predictor%E2%80%93corrector_methodoldid=551354205. Accessed 10 Oct 2013
Zadeh LA (1962) From circuit theory to system theory. Proc IRE 50(5):856–865
Zwillinger D (1998) Handbook of differential equations. Academic, Waltham